DEVELOPER GUIDE

DRAFT SPEC · V0.1 · NOVEMBER 2026

```
POST /v1/projects
  → 201 proj_7f3c…

POST /v1/upload_sessions
  → 201 { "url": … }

delivery.submitted ✓
  ShootPortal-Signature
```

SKYYTASK × SHOOTPORTAL

ShootPortal Platform API

For SkyyTask's developers: authentication, every endpoint, the hosted upload screen, webhooks and signature checks, and going live.

SKYY TASK × ShootPortal

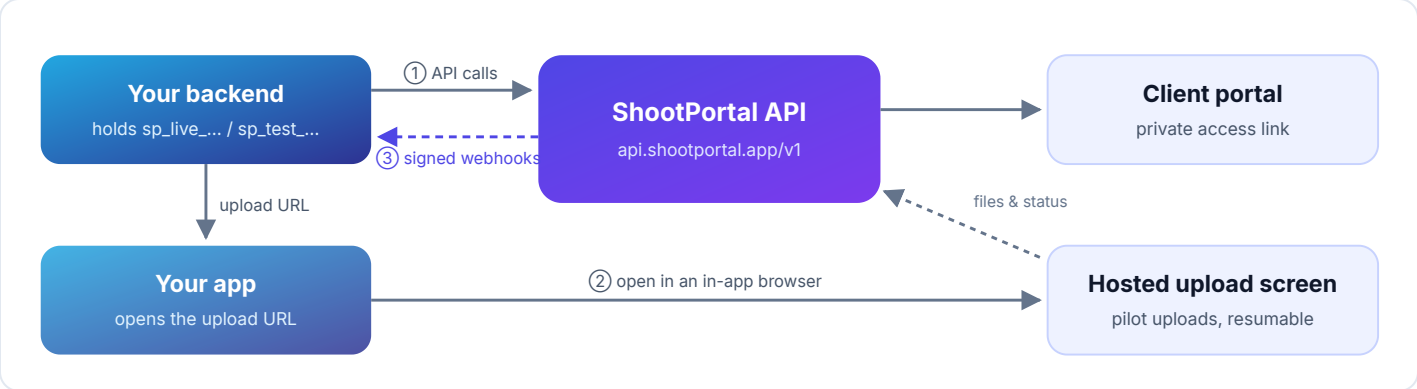
Base URL
`https://api.shootportal.app/v1`

01	**Overview & quickstart**	How the pieces fit, your first calls
02	**Authentication & conventions**	Keys, IDs, idempotency, errors, limits
03	**Endpoint reference**	Every v1 endpoint and object
04	**Upload sessions & embedding**	iOS, Android, cross-platform, web
05	**Deliveries, access links & quotes**	Lifecycle, publishing, pricing
06	**Webhooks**	Events, payloads, signatures, retries
07	**Test vs. live & go-live**	Sandbox, checklist, support

⚠ **Draft specification v0.1. Endpoints, fields and events may change before the sandbox opens. Not yet available.** We'll share the OpenAPI file and a Postman collection with the sandbox.

Three things to build: create, open, listen

Your server uses a secret key. Your app only opens a short-lived upload link. ShootPortal calls your server back with signed webhooks.



A Environments

MODE	KEY PREFIX	WHAT HAPPENS
Test	`sp_test_`	Isolated sandbox. Nothing billed; emails only to allowlisted addresses.
Live	`sp_live_`	Real pilots and clients. Same endpoints, same code.

B Quickstart

1. **Check your key**
   `GET /v1/ping` → `{"account":"SkyyTask","mode":"test"}`
2. **Create a project**
   With your job number as `external_id`.
3. **Add a recipient and a contributor**
   The client and the pilot.
4. **Create an upload session**
   Open the returned `url` on a phone and upload.
5. **Receive webhooks**
   Watch `delivery.submitted` arrive, then verify its signature.

YOUR FIRST CALL

```
# Create a delivery project for job JOB-10482 (test mode)
curl https://api.shootportal.app/v1/projects \
  -H "Authorization: Bearer sp_test_…" \
  -H "Idempotency-Key: job-10482-create" \
  -H "Content-Type: application/json" \
  -d '{
    "external_id": "JOB-10482",
    "title": "123 Demo Street — Aerial photo + video",
    "address": { "line1": "123 Demo Street", "city": "Sampletown",
                 "region": "FL", "postal_code": "00000", "country": "US" }
  }'

# 201 Created
{ "id": "proj_7f3c…", "object": "project", "external_id": "JOB-10482",
  "status": "scheduled", "livemode": false }
```

How every request works

A Authentication

```
Authorization: Bearer sp_live_4eC39HqLyjWDarjtT1zdp7dc
```

- **Server-side only.** Never put a secret key in an app, website or repository.
- **Shown once.** ShootPortal stores only a fingerprint (hash) of each key.
- **Rotate without downtime.** Several keys can be active at once: add the new one, deploy, then revoke the old.
- **Lost or leaked?** Tell us; we revoke it immediately and issue a new one.

B Idempotency (safe retries)

```
Idempotency-Key: job-10482-create
```

- Send a unique key on every **POST**. Retrying with the same key returns the original result, so a timeout never creates a duplicate project.
- Keys are remembered for **24 hours**.
- The same key with a different body returns `409 idempotency_conflict`.
- Tip: build keys from your own IDs, e.g. `job-<number>-create`.

C IDs, your IDs & metadata

- Object IDs are prefixed: `proj_` `rcp_` `ctb_` `ups_` `file_` `dlv_` `lnk_` `evt_`. Treat them as opaque strings.
- Every create accepts your own `external_id` (unique per object type) and a `metadata` map of up to 20 string keys.
- Look up by your ID: `GET /v1/projects?external_id=JOB-10482`, so you never have to store ours.

D Lists, limits & request IDs

- **Pagination:** `?limit=` (max 100) and `starting_after=<id>`. Responses include `has_more`.
- **Rate limit:** 100 requests/minute per key. Above that you get `429` with `Retry-After`.
- **Request-Id** header on every response. Include it when contacting support.
- JSON in and out; timestamps are ISO 8601 UTC; sizes are in bytes.

E Errors

```
{
  "error": {
    "type": "invalid_request_error",
    "code": "parameter_missing",
    "message": "email is required",
    "param": "email",
    "request_id": "req_9a41…"
  }
}
```

HTTP	TYPE	MEANING
400 / 422	`invalid_request_error`	Missing or invalid field (see `param`)
401	`authentication_error`	Missing, wrong or revoked key
403	`permission_error`	API access paused for this account
404	`not_found`	No such object in this mode
409	`idempotency_error` / conflict	Key reused with a new body; state conflict
429	`rate_limit_error`	Slow down; retry after the header's seconds
500	`api_error`	Our side. Safe to retry with the same idempotency key

03 · ENDPOINT REFERENCE

Every v1 endpoint

ENDPOINT	PURPOSE	KEY FIELDS
`GET` `/v1/ping`	Check your key and mode	→ `account`, `mode`
`POST` `/v1/projects`	Create a delivery project	`external_id`, `title`, `address`, `metadata`
`GET` `/v1/projects/{id}`	Retrieve a project	
`GET` `/v1/projects?external_id=`	Find by your ID; list	`limit`, `starting_after`
`PATCH` `/v1/projects/{id}`	Update title, address, metadata	
`POST` `/v1/projects/{id}/recipients`	Add an end client	`email`, `name`, `phone?`, `external_id?`
`DELETE` `/v1/projects/{id}/recipients/{rcp}`	Remove access (revokes links)	
`POST` `/v1/projects/{id}/recipients/{rcp}/access_links`	Mint a private portal link	`expires_in?` (never past file expiry)
`POST` `/v1/projects/{id}/contributors`	Add a pilot (no account needed)	`email`, `name`, `external_id`
`POST` `/v1/upload_sessions`	Get a hosted upload link	`project`, `contributor`, `return_url?`, `expires_in?`
`GET` `/v1/upload_sessions/{id}`	Session status and files so far	
`GET` `/v1/projects/{id}/files`	List uploaded files	section, name, size, type
`GET` `/v1/projects/{id}/deliveries`	List a project's deliveries	
`GET` `/v1/deliveries/{id}`	Status, price, timestamps, counts	
`POST` `/v1/deliveries/{id}/publish`	Publish (when auto-publish is off)	`notify?`
`POST` `/v1/deliveries/quote`	Price a job before it runs	`expected_bytes`

PROJECT

```
{ "id": "proj_7f3c…",
  "object": "project",
  "external_id": "JOB-10482",
  "title": "123 Demo Street — …",
  "status": "scheduled",
  "delivery_mode": "delivery_only",
  "metadata": {},
  "livemode": false,
  "created": "2026-12-15T15:02:11Z" }
```

RECIPIENT

```
{ "id": "rcp_51b0…",
  "object": "recipient",
  "project": "proj_7f3c…",
  "email": "pat.agent@example.com",
  "name": "Pat Agent",
  "external_id": "C-889" }
```

CONTRIBUTOR

```
{ "id": "ctb_2c9d…",
  "object": "contributor",
  "name": "Sam Pilot",
  "external_id": "P-4410" }
```

PROJECT STATUS VALUES

`scheduled`	Created, nothing submitted yet
`ready_for_review`	A delivery was submitted
`delivered`	Published to recipients

Recipients are matched by email, so the same client across many jobs is one person with many private links.

Open ShootPortal's upload screen inside your app

An upload session is a one-job, one-pilot, short-lived link. Create it on your server, hand the `url` to your app, and open it. No key ever reaches the phone.

CREATE A SESSION (SERVER)

```
POST /v1/upload_sessions
Idempotency-Key: job-10482-ups-1

{ "project": "proj_7f3c…",
  "contributor": "ctb_2c9d…",
  "return_url": "skyytask://jobs/JOB-10482/delivered",
  "expires_in": 86400 }

# 201 Created
{ "id": "ups_2b8e…",
  "url": "https://skyytask.shootportal.app/u/ups_2b8e…?t=…",
  "status": "open",
  "expires_at": "2026-12-16T15:05:00Z" }
```

A Session lifecycle

OPEN → SUBMITTED or EXPIRED

- Valid for up to **24 hours**; create a fresh one any time.
- Uploads go directly to storage in chunks and **resume** after drops, reloads or app restarts.
- On submit, the pilot confirms they're finished; you receive `delivery.submitted`.
- Then the screen redirects to `return_url`; on the web it also posts a message.
- Unused and expired: you get `upload_session.expired`.

B Open it in your app

IOS (SWIFT): IN-APP SAFARI VIEW

```
import SafariServices
let url = URL(string: session.url)!
let vc = SFSafariViewController(url: url)
present(vc, animated: true)
// return_url (skyytask://…) reopens your app when done
```

ANDROID (KOTLIN): CHROME CUSTOM TABS

```
CustomTabsIntent.Builder().build()
  .launchUrl(context, Uri.parse(session.url))
// handle skyytask://jobs/… with an intent filter
```

REACT NATIVE / FLUTTER

```
// React Native: react-native-inappbrowser-reborn
InAppBrowser.open(session.url)
// Flutter: url_launcher, LaunchMode.inAppBrowserView
```

WEB: NEW WINDOW + COMPLETION MESSAGE

```
const origin = "https://skyytask.shootportal.app";
window.open(session.url, "upload");
addEventListener("message", (e) => {
  if (e.origin !== origin) return;
  if (e.data.type === "shootportal.upload.submitted")
    refreshJob();
});
```

Use an in-app browser, not a raw web view, for the most reliable file pickers and uploads. Phones pause uploads when the app is backgrounded or the screen locks; they resume automatically on return, and the screen reminds pilots to keep it open. Always treat the **webhook** as the source of truth, not the redirect.

What happens after the pilot taps Submit

1. **submitted** — Size, tier & price fixed
2. **published** — Auto, or via `/publish`
3. **viewed / downloaded** — Per recipient, first time
4. **expired** — After 90 days unless extended

DELIVERY OBJECT

```
{ "id": "dlv_8a2f…",
  "object": "delivery",
  "project": "proj_7f3c…",
  "project_external_id": "JOB-10482",
  "contributor_external_id": "P-4410",
  "status": "published",
  "file_count": 41,
  "total_bytes": 3400000000,
  "tier": "standard",
  "price": { "amount": 300, "currency": "usd",
             "credit_applied": 0, "amount_due": 300 },
  "submitted_at": "2026-12-15T16:40:02Z",
  "published_at": "2026-12-15T16:40:03Z",
  "first_viewed_at": "2026-12-15T17:12:40Z",
  "first_downloaded_at": null,
  "view_count": 3, "download_count": 0,
  "expires_at": "2027-03-15T16:40:02Z" }
```

Prices are in cents. The price is fixed when the delivery is submitted; files added within 7 days join the same delivery and can move it up a tier, but it's never charged twice.

PRIVATE PORTAL LINK FOR A RECIPIENT

```
POST /v1/projects/proj_7f3c…/recipients/rcp_51b0…
     /access_links

# 201 Created
{ "object": "access_link",
  "url": "https://skyytask.shootportal.app/a/lnk_…?t=…",
  "expires_at": "2027-03-15T16:40:02Z" }
```

- ✓ View-only and limited to that project: no login, no code.
- ✓ Valid up to 90 days, never past the files' expiry. Renew by minting again.
- ✓ Removing the recipient revokes all their links.

QUOTE A JOB BEFORE IT RUNS

```
POST /v1/deliveries/quote
{ "expected_bytes": 18000000000 }

{ "tier": "large",
  "price": { "amount": 1200, "currency": "usd" } }
```

Who emails the client?

An account setting: ShootPortal sends a branded email with the link, SkyyTask sends the link itself, or both. Publishing on submit can also be switched to manual (`POST /v1/deliveries/{id}/publish`).

Recap: one job, call by call

#	YOUR BACKEND SENDS	YOU GET BACK
1	`POST` `/v1/projects`	`proj_…` ; or look it up later by `external_id`
2	`POST` `/v1/projects/{id}/recipients` & `/contributors`	`rcp_…` , `ctb_…`
3	`POST` `/v1/upload_sessions` when the pilot taps “Transfer deliverables”	A one-time `url` : open it in your app
4	Nothing; just listen	Webhooks: `delivery.submitted` → `published` → `viewed` → `downloaded`

06 · WEBHOOKS

Signed status updates, retried until you've got them

EVENT	WHEN
`delivery.submitted`	Pilot confirms they're done (includes size & price)
`delivery.published`	Portal is live for recipients
`delivery.viewed`	First view, per recipient
`delivery.downloaded`	First download, per recipient
`upload_session.expired`	Link expired with nothing submitted

EVERY EVENT LOOKS LIKE THIS

```
{ "id": "evt_5c1d…",
  "type": "delivery.submitted",
  "created": "2026-12-15T16:40:02Z",
  "livemode": false,
  "api_version": "v1",
  "data": { "object": { /* the delivery */ } } }
```

A Verify the signature (always)

```
ShootPortal-Signature: t=1765816802,v1=5f8e0c3a…   # v1 = HMAC-SHA256(secret, t + "." + raw_body), hex
```

NODE.JS

```
import crypto from "node:crypto";
export function verify(rawBody, header, secret) {
  const parts = Object.fromEntries(
    header.split(",").map((p) => p.split("=")));
  if (Math.abs(Date.now() / 1000 - Number(parts.t)) > 300)
    return false; // older than 5 minutes
  const expected = crypto.createHmac("sha256", secret)
    .update(parts.t + "." + rawBody).digest("hex");
  return expected.length === parts.v1.length &&
    crypto.timingSafeEqual(Buffer.from(expected), Buffer.from(parts.v1));
}
```

PYTHON

```
import hmac, hashlib, time

def verify(raw_body: bytes, header: str, secret: str) -> bool:
    parts = dict(p.split("=", 1) for p in header.split(","))
    if abs(time.time() - int(parts["t"])) > 300:
        return False  # older than 5 minutes
    signed = parts["t"].encode() + b"." + raw_body
    expected = hmac.new(secret.encode(), signed,
                        hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, parts["v1"])
```

Respond fast

Return any `2xx` within 10 seconds, then do slow work in the background.

Retries for 3 days

1 min, 5 min, 30 min, 2 h, 6 h, then every 12 h. After 3 days of failures the endpoint is paused and we email you.

Ignore duplicates

Delivery is at-least-once. Store each `evt_` ID and skip ones you've already handled.

Use the **raw** request body for verification (before JSON parsing). Your signing secret (`whsec_…`) is shared once when your endpoint is registered; ask us to send a test event at any time.

07 · TEST VS. LIVE & GO-LIVE

Build in the sandbox, launch with one key swap

TEST MODE · SP_TEST_

- A complete, isolated copy of your account: projects, uploads, portal and webhooks all work
- Nothing is ever billed
- Emails go only to addresses you ask us to allowlist; everything else is held in a log we can share
- Try the whole flow with fake data, e.g. job JOB-10482, Sam Pilot, Pat Agent

LIVE MODE · SP_LIVE_

- Real pilots, real clients, real emails
- Same endpoints, same payloads, same code
- Test and live data never mix; IDs from one mode don't exist in the other
- Separate webhook endpoint and signing secret per mode

A Go-live checklist

- ☐ **Keys**: secret keys only on your servers, in a secrets manager
- ☐ **Idempotency**: an `Idempotency-Key` on every POST, built from your IDs
- ☐ **External IDs**: your job, client and pilot IDs on every create
- ☐ **Upload screen**: opens in an in-app browser; `return_url` handled
- ☐ **Real-phone test**: a multi-GB upload interrupted and resumed
- ☐ **Webhooks**: signatures verified on the raw body; duplicates ignored
- ☐ **All five events handled**, including `upload_session.expired`
- ☐ **Errors**: `request_id` logged; 429 and 5xx retried with backoff
- ☐ **Switch**: live key and live webhook secret deployed; test data left behind

B Versioning

The version is in the URL (`/v1`). Within v1 we only make **additive** changes: new endpoints, new optional fields, new event types. Write your code to ignore fields and events it doesn't know. Breaking changes would ship as `/v2` , with advance notice and an overlap period.

C Support

Jackson Bridges
Founder, ShootPortal

jackson@shootportal.app
(662) 687-1259

Include the **Request-Id** (or the `evt_` ID for webhooks) and we can see exactly what happened.

Coming with the sandbox
OpenAPI 3.1 file · Postman collection · test keys · a demo partner that runs the full flow end to end